

Why LangGraph?

Module 1 : The problem with chains

Before LangGraph, the standard way to build LLM applications was a **chain** — you write a prompt, pass it to an LLM, get output. Done.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-20250514")
prompt = ChatPromptTemplate.from_template("Answer this: {question}")

chain = prompt | llm
result = chain.invoke({"question": "What is Python?"})
```

This is clean, simple, and works well — **for simple tasks**.

The moment your task needs any of the following, a chain breaks down:

- **Loops** — do something, check the result, do it again if needed
- **Branching** — take different paths based on what the LLM decides
- **Tool use** — call an API or run code, then continue reasoning based on the result
- **Memory** — remember what happened in previous turns

Chains are linear. Input → LLM → Output. No going back. No branching. No state.

What an agent actually is ?

An agent is a system that operates in a loop:

1. **Think** — the LLM looks at the current situation
2. **Decide** — should I use a tool? Am I done? What next?
3. **Act** — call a tool, write a response, ask a question
4. **Observe** — see the result of the action
5. **Repeat** — go back to step 1 until the task is complete

This is fundamentally different from a chain. The LLM isn't just producing an output — it's making decisions that control what happens next.

LangGraph is "stateful"

A stateless system has no memory between operations. Every time you call it, it starts fresh. It has no idea what happened before.

A regular Python function is stateless:

```
def add(a, b):  
    return a + b  
  
add(2, 3) # returns 5  
add(2, 3) # returns 5 again – it remembers nothing
```

A basic LLM API call is also stateless:

```
import anthropic  
client = anthropic.Anthropic()  
  
# Call 1  
client.messages.create(  
    model="claude-sonnet-4-20250514",  
    max_tokens=100,  
    messages=[{"role": "user", "content": "My name is Arjun"}]  
)  
  
# Call 2 – the model has zero memory of Call 1  
client.messages.create(  
    model="claude-sonnet-4-20250514",  
    max_tokens=100,  
    messages=[{"role": "user", "content": "What is my name?"}]  
)  
# Response: "I don't know your name."
```

The LLM itself is stateless. Every API call is completely independent. It knows nothing about what came before unless you explicitly pass that history in.

A **stateful** system remembers things across steps.

It maintains a shared "workspace or STATE" that accumulates information as the system runs.

LangGraph is stateful because it maintains a **State object** — a shared dictionary TypedDict that every node in the graph can read from and write to. This State persists and evolves as execution moves from node to node.

Node = process / task

Old state (as input) ----> Node ---> Updated State (as output)

{dict} ---> node ---> {dict}

```
from typing_extensions import TypedDict
from typing import Annotated
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages] # conversation history
    user_name: str # remembered across steps
    task_complete: bool # tracks progress
    retry_count: int # knows how many attempts made
```

This State object is the "memory" of the entire graph. Every node sees the same State. Every node's updates are merged back into it. Nothing is lost between steps.

3-node graph, notice how state accumulates across all three nodes:

Node : old state as input and returns updated state as output

```
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

...
state = {"number" : int ,
        "history" : list}
...

class State(TypedDict):
    number: int
    history: list

def step_one(state: State) -> dict:
    print(f"Step 1 sees number: {state['number']}")
    return {
        "number": state["number"] + 10,
        "history": ["step_one ran"]
    }
...
first node output :
{
"number" : 30,
"history" : ["step_one ran"]
}
...
```

```

def step_two(state: State) -> dict:
    # state["number"] is already 110 here – it sees step_one's update
    print(f"Step 2 sees number: {state['number']}")
    return {
        "number": state["number"] * 2,
        "history": ["step_two ran"]
    }

...
second node output :
{
    "number" : 60,
    "history" : ["step_one ran","step_two_ran"]
}
...

def step_three(state: State) -> dict:
    # state["number"] is 220 here – it sees step_two's update
    print(f"Step 3 sees number: {state['number']}")
    return {
        "number": state["number"] - 5,
        "history": ["step_three ran"]
    }

builder = StateGraph(State)
builder.add_node("step_one", step_one) #syntax : graph.add_node("nameofnode-
user defined",function)
builder.add_node("step_two", step_two)
builder.add_node("step_three", step_three)

builder.add_edge(START, "step_one")
builder.add_edge("step_one", "step_two")
builder.add_edge("step_two", "step_three")
builder.add_edge("step_three", END)

graph = builder.compile()

result = graph.invoke({"number": 100, "history": []})

print(result)
# {
#     "number": 215,                ← 100 → 110 → 220 → 215, accumulated across 3
nodes
#     "history": ["step_one ran", "step_two ran", "step_three ran"]
# }

```

Each node picked up exactly where the previous node left off. That's statefulness in action.

Why is this critical for agentic systems?

Think about what an agent needs to do its job:

1. Remember the conversation

An agent handling a multi-turn conversation needs to know what was said 5 messages ago. Without state, every response is a fresh start — the agent would forget the user's name, their problem, what was already tried.

python

```
class State(TypedDict):
    messages: Annotated[list, add_messages]

# Message 1 added to state → Message 2 added → Message 3 added
# By message 10, the agent has the full conversation in state["messages"]
# Every node can read that history at any point
```

2. Track what it has already done

An agent that searches the web needs to know which queries it already ran, so it doesn't repeat them. An agent fixing code needs to know how many attempts it has made, so it can give up after 3 failures instead of looping forever.

python

```
class State(TypedDict):
    messages: Annotated[list, add_messages]
    searches_done: list      # don't repeat these
    retry_count: int         # stop after 3
    task_status: str        # "in_progress" | "done" | "failed"
```

3. Pass information between nodes

One node searches the web and stores results. A completely different node, running later, needs those results to write a report.

Without shared state, there is no way to pass data between nodes except messy function arguments.

python

```
def search_node(state: State) -> dict:
    results = search_web(state["query"])
    return {"search_results": results}    # stored in state

def writer_node(state: State) -> dict:
    # reads what search_node stored – no direct connection needed
    results = state["search_results"]
    report = llm.invoke(f"Write a report using: {results}")
    return {"final_report": report}
```

4. Make decisions based on accumulated context

A conditional edge (branching logic) reads the current state to decide what happens next. Without state, there's nothing to branch on.

python

```
def should_continue(state: State) -> str:
    # This decision is only possible because state tracks history
    if state["retry_count"] >= 3:
        return "give_up"
    if state["task_status"] == "done":
        return "end"
    return "try_again"
```

The precise mechanism — how LangGraph maintains state

It's important to understand exactly what's happening under the hood:

1. You call `graph.invoke(initial_state)`
2. LangGraph creates a **state snapshot** from your initial input
3. A node runs — it receives the current state snapshot, does its work, returns a **partial update** (just the keys it changed)
4. LangGraph **merges** that partial update into the state using reducers.
5. The next node runs with the updated state.
6. This continues until the graph reaches END.

The key insight: **nodes never modify state directly.**

They return a dictionary of updates. LangGraph does the merging.

This is intentional — it makes the system predictable, debuggable, and safe to run in loops.

```
# A node does NOT do this:
state["messages"].append(new_message) # WRONG — never mutate directly

# A node does this:
return {"messages": [new_message]} # RIGHT — return updates, LangGraph
merges
```

Module 2 — Nodes, Edges, and State

These are the only three things LangGraph is made of. Every agent, no matter how complex, is just these three concepts combined. We'll go deep on each one.

Part 1 — State

We touched on State in Module 1. Now let's understand it fully.

State is a `TypedDict` — a Python dictionary with typed keys. It is the single shared workspace for your entire graph. Every node reads from it.

Every node writes back to it. It is the only way information moves between nodes.

```
from typing_extensions import TypedDict

class State(TypedDict):
    messages: list
    user_name: str
    step_count: int
    is_done: bool
```

That's it. No magic. Just a typed dictionary.

Rules about State:

- You define it once, at the top
- Every node receives the full current state as its input
- Every node returns only the keys it wants to update — not the full state
- LangGraph merges the returned updates into the existing state
- Keys you don't return are untouched

```
class State(TypedDict):
    name: str
```

```
age: int
score: int

def my_node(state: State) -> dict:
    # I only care about score – I return only score
    # name and age are untouched in state
    return {"score": state["score"] + 1}
```

What goes inside State?

Anything your nodes need to share. Common patterns:

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

# A chatbot agent
class ChatState(TypedDict):
    messages: Annotated[list, add_messages] # conversation history
    user_name: str # remembered context

# A research agent
class ResearchState(TypedDict):
    query: str # what to research
    search_results: list # raw results from web search
    final_report: str # written output
    searches_done: int # loop counter

# A code debugging agent
class DebugState(TypedDict):
    code: str # original code
    error: str # current error message
    fixed_code: str # agent's proposed fix
    attempts: int # how many fixes tried
    solved: bool # are we done?
```

Design your State around the question: **what does every node need to know to do its job?**

Part 2 — Nodes

A node is a plain Python function. Nothing special about it. It takes State as input and returns a dictionary of updates.

python

```
def my_node(state: State) -> dict:
    # 1. Read whatever you need from state
    # 2. Do your work (call LLM, run a tool, compute something)
    # 3. Return only the keys you want to update
    return {"some_key": some_value}
```

That's the complete contract. Every node in every LangGraph system follows this exact signature.

Three types of nodes :

Type 1 — LLM node (the most common)

Calls an LLM and stores the response in state.

```
from langchain_anthropic import ChatAnthropic

llm = ChatAnthropic(model="claude-sonnet-4-20250514")

def agent_node(state: State) -> dict:
    # Pass the conversation history to the LLM
    response = llm.invoke(state["messages"])
    # Return the new message — add_messages reducer appends it
    return {"messages": [response]}
```

Type 2 — Tool node (executes an action)

Calls an external service — web search, database, API, code runner — and stores the result.

```
import requests

def search_node(state: State) -> dict:
    query = state["query"]
    # Call a real search API here
    results = requests.get(f"https://api.search.com?q={query}").json()
    return {"search_results": results["items"]}
```

Type 3 — Logic node (pure computation, no LLM)

Updates state based on computation — increments a counter, formats data, checks a condition.

```
def increment_attempts(state: State) -> dict:
    return {"attempts": state["attempts"] + 1}

def format_report(state: State) -> dict:
    raw = state["search_results"]
    formatted = "\n".join([r["title"] for r in raw])
    return {"formatted_results": formatted}
```

What a node receives vs what it returns

A node receives — the full state.

Every single key. Everything in the state dictionary, with all the latest values at that point in time.

```
class State(TypedDict):
    messages: list
    user_name: str
    attempts: int
    is_done: bool

def my_node(state: State) -> dict:
    # state here contains ALL four keys
    print(state["messages"])    # works
    print(state["user_name"])    # works
    print(state["attempts"])    # works
    print(state["is_done"])     # works
```

No matter how many keys are in your State, the node gets all of them.

A node returns — only what it changed.

You don't return the full state back. You only return the keys you actually modified. Everything else stays untouched automatically.

```
def my_node(state: State) -> dict:
    # I only care about attempts – I only return attempts
    return {"attempts": state["attempts"] + 1}

    # messages, user_name, is_done – I didn't touch them
    # LangGraph keeps their existing values automatically
```

That's what "partial update" means. Not the full state — just the slice you changed.

Why does this design exist?

Because nodes are specialized. In a real agent:

- The `agent` node only cares about messages
- The `counter` node only cares about attempts
- The `search` node only cares about search results

If every node had to return the full state, every node would need to know about every key — even the ones it has nothing to do with. That would make nodes tightly coupled and fragile.

Partial returns keep nodes independent.

LangGraph's job after a node runs

After your node returns its partial update, LangGraph does this:

```
old state + partial update = new full state
```

Concretely:

python

```
# State before node runs
{
    "messages": ["hello"],
    "user_name": "Arjun",
    "attempts": 2,
    "is_done": False
}

# Node returns this partial update
{
    "attempts": 3
}

# LangGraph merges them – new state passed to next node
{
    "messages": ["hello"],      # unchanged
    "user_name": "Arjun",      # unchanged
    "attempts": 3,              # updated
}
```

```
"is_done": False          # unchanged
}
```

The next node receives this fully merged state. It sees `attempts` as 3. It never needs to know that only `attempts` changed — it just sees the current complete picture.

Let's trace through exactly what each node receives and returns:

```
class State(TypedDict):
    name: str
    score: int
    grade: str

def node_a(state: State) -> dict:
    # Receives: {"name": "Arjun", "score": 0, "grade": ""}
    print(state)

    return {"score": 100}
    # Returns only score. name and grade untouched.

def node_b(state: State) -> dict:
    # Receives: {"name": "Arjun", "score": 100, "grade": ""}
    # Notice score is already 100 – it sees node_a's update
    print(state)

    return {"grade": "A"}
    # Returns only grade. name and score untouched.

def node_c(state: State) -> dict:
    # Receives: {"name": "Arjun", "score": 100, "grade": "A"}
    # It sees everything accumulated from node_a and node_b
    print(state)

    return {"name": state["name"] + " (graded)"}
    # Returns only name.

# Final state after all three nodes:
# {"name": "Arjun (graded)", "score": 100, "grade": "A"}
```

Each node got the full picture. Each node only returned what it changed. LangGraph silently merged everything in between.

Registering nodes with the graph

Writing the function is step one. You also need to register it with the graph builder:

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(State)

# Register each node – first arg is the name (string), second is the
# function
builder.add_node("agent", agent_node)
builder.add_node("search", search_node)
builder.add_node("format", format_report)
```

The name you give the node (the string) is what you use when defining edges. The function is what actually runs.

Part 3 — Edges

Edges are the rules that control which node runs next. This is where the "graph" in LangGraph earns its name.

There are two types of edges.

Type 1 — Normal edge

Always goes from node A to node B. No conditions. No decisions. Every time node A finishes, node B runs next.

python

```
builder.add_edge("search", "format")      # after search, always run format
builder.add_edge("format", "agent")       # after format, always run agent
builder.add_edge(START, "agent")          # agent is always the first node
builder.add_edge("agent", END)            # agent is always the last node
```

Use normal edges when the flow is fixed and predictable.

Type 2 — Conditional edge

Reads the current state and decides which node to go to next. This is how agents make decisions.

A conditional edge needs a **routing function** — a plain Python function that takes state and returns a string (the name of the next node).

python

```
def route(state: State) -> str:
    # Read state, return the name of the next node
    if state["attempts"] >= 3:
        return "give_up"
    if state["solved"]:
        return "end"
    return "try_again"

# Register the conditional edge
builder.add_conditional_edges(
    "agent",      # from this node
    route,        # call this function to decide
)
```

LangGraph calls `route(state)` after `agent` finishes, looks at what it returns, and routes to that node. That's the entire mechanism.

The two special nodes — START and END

LangGraph provides two built-in nodes you don't define yourself:

- `START` — the entry point. Add an edge from `START` to whichever node should run first.
- `END` — the exit point. Add an edge to `END` from whichever node should run last. When a node routes to `END`, the graph stops.

python

```
from langgraph.graph import START, END

builder.add_edge(START, "agent")    # agent runs first
builder.add_edge("agent", END)      # graph stops after agent
```

Putting it all together — a complete working example

Here is a real agent that loops. It has an LLM node that calls Claude, and it keeps running until the LLM decides it is done. Every concept from this module is present.

python

```
from langgraph.graph import StateGraph, START, END
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import HumanMessage, SystemMessage
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

# --- State ---
class State(TypedDict):
    messages: Annotated[list, add_messages]
    attempts: int

# --- LLM setup ---
llm = ChatAnthropic(model="claude-sonnet-4-20250514")

# --- Nodes ---
def agent_node(state: State) -> dict:
    response = llm.invoke(state["messages"])
    return {
        "messages": [response],
        "attempts": state["attempts"] + 1
    }

# --- Routing function ---
def should_continue(state: State) -> str:
    last_message = state["messages"][-1]

    # If LLM's response contains "DONE", stop
    if "DONE" in last_message.content:
        return "end"

    # Safety – stop after 5 attempts no matter what
    if state["attempts"] >= 5:
        return "end"

    # Otherwise keep going
    return "continue"

# --- Build graph ---
```

```

builder = StateGraph(State)

builder.add_node("agent", agent_node)

builder.add_edge(START, "agent")

builder.add_conditional_edges(
    "agent",
    should_continue,
    {
        "continue": "agent",    # loop back to agent
        "end": END              # stop the graph
    }
)

graph = builder.compile()

# --- Run ---
initial_state = {
    "messages": [
        SystemMessage(content="You are a helpful assistant. When you are
finished helping, say DONE."),
        HumanMessage(content="Count from 1 to 3, one number per message.")
    ],
    "attempts": 0
}

result = graph.invoke(initial_state)

for msg in result["messages"]:
    print(msg.content)

```

Walk through what happens here step by step:

1. Graph starts, routes from `START` to `agent`
2. `agent_node` calls Claude, gets response "1", appends it to messages, increments attempts to 1
3. `should_continue` checks state — no "DONE" in response, attempts is 1 — returns "continue"
4. Graph loops back to `agent`
5. `agent_node` calls Claude again, gets "2", appends, attempts is 2
6. `should_continue` — no "DONE", attempts 2 — returns "continue"
7. Loop again — Claude says "3. DONE"
8. `should_continue` finds "DONE" — returns "end"

9. Graph routes to `END` , execution stops
10. `result` contains the full final state with all messages